function uses Firestore's `getCountFromServer` for efficient counting without downloading entire documents.

8.9 Storage Service

The storage service handles file uploads to Cloudinary for waste report photos and delivery proofs:

```
const CLOUD_NAME = process.env.NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME;
const UPLOAD_PRESET = process.env.
   NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET;

export async function uploadImage(
  file: File
): Promise<string> {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('upload_preset', UPLOAD_PRESET || '');
  formData.append('folder', 'agritrace');

  const response = await fetch(
    `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
    { method: 'POST', body: formData }
  );

  if (!response.ok) {
    throw new Error('Upload failed');
  }
  const data = await response.json();
  return data.secure_url;
}

export async function uploadMultipleImages(
  files: File[]
): Promise<string[]> {
  const uploadPromises = files.map((file) => uploadImage(file));
  return Promise.all(uploadPromises);
}
```

Listing 8.9: Storage Service (storage-service.ts)

Image uploads use Cloudinary's unsigned upload preset mechanism, which avoids exposing API secrets on the client side. The `uploadMultipleImages` function leverages `Promise.all` for concurrent uploads, minimizing total upload time when farmers submit multiple waste pho-